

Two objects

a random walk on the tree of activation codes, an isotropic process on its Cantor set,
and activation means as Dixmier traces

Working note VIII for the continuation-algebra programme
self-reviewed draft, not independently verified

3 October 2026

Abstract

The proposal has two separate objects.

- *Object 1*: nodes are binary strings (activation codes), and arrows are shift operators with self-loops, so the structure is a groupoid with a convolution algebra.
- *Object 2*: the tree of codes, carrying values defined from the weights, whose Cantor-set distance (last point of contact) should translate into a process, and a run length, for estimating activation means. The Dixmier trace and the zeta function are expected to play a role.

Using Bendikov–Grigor’yan–Pittet–Woess (BGPW) on isotropic Markov semigroups on ultrametric spaces, and the path correspondences of quantum graphs (Khatua–Roy), we make this exact.

(1) Object 1 generates object 2. The de Bruijn code graph, with self-loops at 0^n and 1^n , is a quantum graph whose path correspondences are the levels of the code tree. Its convolution algebra is the Cuntz algebra $\mathcal{O}_2$: nodes are projections $P_v = S_v S_v^*$ indexed by binary strings, and arrows are the shift isometries S_0, S_1 .

(2) A walk on object 1 samples object 2. A nearest-neighbour walk on the code tree that backtracks with depth-dependent probability, and otherwise steps forward with the conditional code law, exits with *exactly* the code law μ . Verified: $\chi^2/\text{dof} = 1.02$ over 64 cylinders.

(3) Duality (BGPW, Kigami). The induced process on the Cantor set is isotropic. Its jump kernel depends only on the last point of contact $x \wedge y$, and its spectrum is $\{1/G(v, o)\}$, given by the Green function of the walk on object 1.

(4) Weights $\rightarrow \kappa \rightarrow$ zeta and Dixmier. With the ultrametric $d(x, y) = \mu(x \wedge y)^{1/s}$, built from the weight-induced code law, the isotropic Laplacian L_s has spectral zeta function $\zeta_s(t) = \text{Tr } L_s^{-t}$. Its simple pole is at $t = s$, with *residue s/h , where h is the entropy rate of the code process*. Its Dixmier trace is integration against μ , so *activation means are Dixmier traces*. The residue and the subtree ratios are checked numerically, the latter to 3×10^{-4} .

(5) Run length. The process runs for time $T(\varepsilon) = 2\varepsilon^{-2} \sum_v G(v, o) e_v$, a Green-function-weighted energy.

(6) The verdict. A ReLU network on Gaussian input has no internal randomness, so every fresh code costs a fresh forward pass. Code cells chosen from the weights carry at most about 10% of the variance at any layer: stratification gains are $\leq 1.10\times$. The two-phase criterion $R_\ell^2 > \ell/L$ fails everywhere. The framework gives an exact NCG formula for the means, but not a faster estimator for random He networks.

1 Object 1: the code graph and its algebra

Read the network's neuron signs sequentially, neuron by neuron and layer by layer. Each input gives a binary string, a path in the binary *code tree* T , of depth nL . The window of the last n signs moves by the de Bruijn shift $s_1 \cdots s_n \mapsto s_2 \cdots s_n b$. After n shifts it holds the next layer's code; the position within the layer is a phase modulo n , and this is “shift modulo cyclicity”. The constant codes 0^n (a dead layer) and 1^n (a fully active layer) carry self-loops.

Proposition 1.1 (object 1 builds object 2). *Take the de Bruijn graph on $\{0,1\}^n$, viewed as a classical quantum graph $(C(\{0,1\}^n), \psi, A)$ with uniform ψ and 0/1 adjacency A . Its k -path correspondences are $E^k = C(\{\text{paths of length } k\}) \cong C(\{0,1\}^{n+k})$ (Khatua–Roy, §4.1), i.e. the levels of the binary tree. The infinite path space is the Cantor set $\{0,1\}^{\mathbb{N}}$ with the one-sided shift. The associated Cuntz–Krieger algebra is the Cuntz algebra $\mathcal{O}_2$, because the edge shift is the full 2-shift. In it, nodes are the projections $P_v = S_v S_v^*$ indexed by binary strings v , and arrows are the isometries S_0, S_1 . A nonnegative function on arrows of the shift groupoid that sums to one along the arrows leaving each point, $\sum_b p(b \mid \cdot) = 1$, acts by the Ruelle transfer operator $f \mapsto \sum_b p(b \mid \cdot) f(\cdot b)$: convolution is one Markov step.*

Paths in object 1 are vertices of object 2. Object 1's Markov steps are transfer operators, which are the moves of a walk on the tree.

2 Object 1's walk samples object 2

Theorem 2.1 (exit measure). *Let μ be any probability on the code Cantor set with $\mu(v) > 0$ for every vertex. Let $0 < q_k < 1/2$ for $k \geq 1$ and $q_0 = 0$. The nearest-neighbour walk on T steps back to the parent with probability $q_{|v|}$, and otherwise moves to the child vb with probability $(1 - q_{|v|}) \mu(vb)/\mu(v)$. It is transient, and its exit measure from the root is exactly μ .*

Proof. The depth process is a birth–death chain that does not depend on μ and drifts outward. So the probability ϵ_k of never returning to the parent, starting from depth k , depends only on k . The walk exits through the cylinder $[v]$ iff its last departure from v goes to some child and never returns. That child is vb with probability proportional to $(1 - q_{|v|}) \frac{\mu(vb)}{\mu(v)} \epsilon_{|v|+1}$, i.e. to $\mu(vb)/\mu(v)$. Induct on depth. $\square$

Check (`code/tree_walk_exit.py`). On a random binary μ of depth 6, with $q_k = 0.3 + 0.1 \sin k$ and 4×10^5 walks, the maximum error is 9×10^{-4} and $\chi^2/\text{dof} = 1.02$ over 64 cylinders.

3 Duality: the process on object 2

Theorem 3.1 (BGPW Thms. 3.8, 7.1, 7.5; Kigami). *The boundary process induced by a transient nearest-neighbour walk on T , with reference measure its exit measure μ , is an isotropic jump process on ∂T : its jump kernel is $J(x, y) = j(x \wedge y)$, a function of the last point of contact only. Its Laplacian has pure point spectrum $\{1/G(v, o)\} \cup \{0\}$, given by the Green function of the walk. The eigenfunctions are $f_C = \mathbf{1}_C/\mu(C) - \mathbf{1}_B/\mu(B)$ for C a child of B , with eigenvalue determined by B . Conversely, every isotropic process, for any reference measure and any ultrametric, arises in this way from a unique walk on T , up to a linear time change.*

The eigenspaces $H_B = \text{span}\{f_C : C \prec B\}$ are the detail spaces of note VII. That note's spectral formula for last-point-of-contact kernels is the finite-tree, discrete-time case of BGPW's Theorem 3.8.

4 Weights, zeta, Dixmier: activation means as Dixmier traces

The weights determine the code law μ under Gaussian input. Given $s > 0$, take the ultrametric

$$d_s(x, y) = \mu(x \wedge y)^{1/s} \quad (\text{Michon weights } \kappa = \mu^{1/s}).$$

By Theorem 3.1, the BGPW isotropic Laplacian L_s of (d_s, μ) has eigenvalue $\mu(B)^{-1/s}$ on H_B , up to the normalization of the distance distribution, and is realized by a unique walk on object 1.

Theorem 4.1 (zeta, entropy, Dixmier). *Suppose the code law is a finite-memory (de Bruijn) Markov process with positive entropy rate h . The general sequential code law is not of this form; the theorem applies to its Markov approximation, or to the infinite-depth limit when that limit is Gibbs. On the binary tree, let*

$$\zeta_s(t) = \text{Tr} (L_s^{-t} |_{1^\perp}) = \sum_B \mu(B)^{t/s}.$$

- (i) ζ_s converges for $t > s$ and has a simple pole at $t = s$, with $\text{Res}_{t=s} \zeta_s = s/h$.
- (ii) For a vertex v , the subtree series $\zeta_{s,v}(t) = \sum_{B \subseteq [v]} \mu(B)^{t/s}$ satisfies $\text{Res} \zeta_{s,v} / \text{Res} \zeta_s = \mu(v)$.
- (iii) Hence, for continuous f on the code Cantor set,

$$\int f d\mu = \lim_{t \downarrow s} \frac{\text{Tr} (M_f L_s^{-t})}{\text{Tr} (L_s^{-t})} = \frac{\text{Tr}_\omega (M_f L_s^{-s})}{\text{Tr}_\omega (L_s^{-s})}.$$

The Dixmier trace of L_s^{-s} is integration against the code law.

Proof. With $\sigma = t/s$ and P the transition matrix on de Bruijn states, $\sum_{|B|=k} \mu(B)^\sigma = \pi_\sigma^\top P_\sigma^{k-m} \mathbf{1}$, where P_σ is the entrywise power. The Perron root $\rho(\sigma)$ is analytic near $\sigma = 1$, with $\rho(1) = 1$ and

$$\rho'(1) = \pi^\top (P \circ \log P) \mathbf{1} = -h.$$

So $\sum_k \rho(\sigma)^k \sim 1/(h(\sigma - 1))$, which gives (i); the factor s comes from $\sigma - 1 = (t - s)/s$.

For (ii), the subtree series starts from the state of v with mass $\mu(v)$. At $\sigma = 1$ the right Perron vector is $\mathbf{1}$, so the residue is $\mu(v)s/h$.

For (iii), H_B is one-dimensional, with a unit eigenvector ϕ_B supported in B . Then $\text{Tr}(M_f L_s^{-t}) = \sum_B \mu(B)^{t/s} \langle \phi_B, f \phi_B \rangle$, and $\langle \phi_B, f \phi_B \rangle$ is a weighted average of f over B . As $t \downarrow s$ the deep B dominate, so (ii) and continuity of f give the limit. The Dixmier form is Connes' residue identification, valid for a simple pole. $\square$

Check (`code/zeta_dixmier.py`). Memory-3 chain, $h = 0.5865$ nats, $s = 1.7$. As $t \downarrow s$, $(t - s)\zeta_s(t) = 2.697, 2.816, 2.857, 2.878$, against $s/h = 2.899$ with $O(t - s)$ convergence. The subtree residue ratios match $\mu(v)$ to 3×10^{-4} .

Corollary 4.2 (activation means). *Let $f_j(c) = \mathbb{E}[F_j \mid \text{code} = c]$ be the conditional mean of output j on the tile with full code c (note V). Then $\mathbb{E}F_j = \int f_j d\mu$ is the Dixmier trace of M_{f_j} against L_s^{-s} . The zeta residue sees only the entropy rate of the code process. Within-tile variation, $\mathbb{E} \text{Var}(F_j \mid \text{code})$, is invisible to the code tree.*

5 Which process, how long, and what it costs

Proposition 5.1 (run length). *Run the stationary isotropic process for time T and average f . The variance is asymptotically*

$$\frac{2}{T} \langle f, L^{-1} f \rangle = \frac{2}{T} \sum_B G(B, o) e_B, \quad e_B = \|P_{H_B} f\|^2.$$

So reaching accuracy ε takes time $T(\varepsilon) = 2\varepsilon^{-2} \sum_B G(B, o) e_B$, the Green-function pairing of the activation's tree energy with object 1's walk.

Cost. A jump with last point of contact at depth k needs a fresh code below k . For a network whose only randomness is its input, that means a fresh input conditioned on the shared prefix, which costs at least one pass through the later layers plus conditional sampling. By note VII (positive spectrum), the process is then never better per evaluation than independent inputs. The only lever is stratification by code cells, priced next.

Proposition 5.2 (two-phase stratification along the code tree). *Let the codes at depth ℓ explain a fraction R_ℓ^2 of $\text{Var } F$ through their cells, and cost a fraction c_ℓ of a forward pass to compute. Run n_1 inputs to depth ℓ to estimate cell masses, and finish $n_2 \leq n_1$ of them. The best variance-times-cost gain over Monte Carlo is*

$$G_\ell = \left(\sqrt{R_\ell^2 c_\ell} + \sqrt{(1 - R_\ell^2)(1 - c_\ell)} \right)^{-2} \quad \text{if } R_\ell^2 > c_\ell, \quad G_\ell = 1 \text{ otherwise.}$$

Proof. The variance is $\approx (1 - R^2) \text{Var } F / n_2 + R^2 \text{Var } F / n_1$, and the cost is $n_1 c + n_2(1 - c)$. Minimize under $n_2 \leq n_1$; the constraint binds exactly when $R^2 \leq c$. $\square$

Measurement (`code/code_tree.energy.py`). Width 256, 2^{20} inputs, eight live outputs. The table gives the stratification gain $\text{Var } F / \mathbb{E} \text{Var}(F \mid \text{cell}) = 1/(1 - R^2)$ when cells are the activation codes of k neurons chosen from the weights. At layer 1 the neurons are ranked by linearized sensitivity of the output; in the middle layer, by the linearized map; at layer $L - 1$, by $|W_L[j, i]|$.

	layer 1			middle			layer $L - 1$		
depth	$k=4$	8	12	4	8	12	4	8	12
4	1.025	1.041	1.056	1.017	1.035	1.057	1.042	1.071	1.100
8	1.006	1.012	1.020	1.020	1.040	1.058	1.043	1.073	1.101
16	1.003	1.006	1.010	1.015	1.023	1.031	1.048	1.073	1.094

At most about 9% of the variance is explained, and it is at the last hidden layer, where $c_\ell = (L-1)/L$. The cheap first-layer cells explain about 1% at depth 16. Hence $G_\ell \approx 1$ everywhere.

6 Verdict

- *The construction is exact.*
 - Object 1 (the de Bruijn code graph, with Cuntz algebra $\mathcal{O}_2$) generates object 2 (the code tree and its Cantor set) through path correspondences.
 - A walk on object 1 samples object 2's code law exactly (Theorem 2.1).
 - The induced process on object 2 is BGPW-isotropic, with last-point-of-contact jumps and Green-function spectrum.

- Weights define $\kappa = \mu^{1/s}$, whose zeta function has residue s/h and whose Dixmier trace is the code law, so activation means are Dixmier traces.
- The run length is a Green-weighted energy.
- *It does not beat Monte Carlo for random He networks.* Randomness enters only at the input, so every fresh code costs a forward pass, and code cells carry little of the variance (above). Together with notes II and VII, the variance of deep random networks is spread over exponentially many cells and high Hermite degrees.
- *Where it would pay.* The structure pays when moving deep in the code tree is cheaper than a full pass. That holds for networks with internal randomness (dropout, noisy units, stochastic depth), where backtracking reuses the prefix computation, and for networks whose code law has low entropy rate h , which concentrates the Dixmier measure.

A Scripts

`code/tree_walk_exit.py` (Theorem 2.1); `code/zeta_dixmier.py` (Theorem 4.1); `code/code_tree_energy.py` with output (Proposition 5.2 inputs).